



MATHÉMATIQUES
VISION
APPRENTISSAGE

PROBABILISTIC GRAPHICAL MODEL

Denoising Score Matching and Diffusion Models

PLANCHON ROMAIN
ARTHUR GILLES
KEVIN WARDAKHAN

SUPERVISORS : PIERRE LATOUCHE, PIERRE-ALEXANDRE MATTEI

1 Introduction

Deep generative models have achieved impressive results on complex datasets. Among them, score-based models form a unifying family : rather than learning a normalized density, they approximate the score function, $\nabla_x \log p(x)$, and use this vector field to transport noise into realistic samples. This perspective unifies Noise Conditional Score Networks (NCSN) [4], Denoising Diffusion Probabilistic Models (DDPM) [2], and the SDE formulation of score-based generative modeling [5].

In this project, we implement and study score-based generative modeling from both discrete and continuous perspectives. Our contributions are the following.

- We illustrate the limitations of single-scale Langevin dynamics on a challenging 2D Gaussian mixture with access to the *exact* score, and show how Annealed Langevin Dynamics (ALD) improves mixing and mode proportions.
- We implement a VE-SDE score model with a U-Net backbone conditioned on the noise level/time, trained with the continuous denoising score matching objective from the SDE framework [5].
- We implement and compare three VE-SDE samplers : P-only, C-only and predictor-corrector (PC).
- We study the impact of EMA regularization
- We study the impact of an SNR-based step-size rule for the Langevin corrector on MNIST and CelebA (64×64).

After each of us successfully reimplemented our own version of NCSN and then the VE-SDE framework, we jointly thought of different experiments, which were then implemented by one of us before a shared analysis : Arthur focused on the Gaussian experiment, Kevin on the CIFAR-10 adaptation and the impact of EMA regularization, and Romain on CelebA and the SNR-based corrector.

2 Genesis of Noise-Conditional Score Models

In this section, we revisit the main ideas that led to the introduction of NCSN by Song and Ermon [4], starting from early insights on score estimation and progressing toward annealed sampling procedures that address key limitations of Langevin dynamics.

2.1 From Score Estimation to Sampling

The concept of estimating the score function $\nabla_x \log p(x)$ without computing the normalizing constant was introduced by Hyvärinen (2005) [3] via *score matching*. The objective avoids computing the density itself and instead minimizes the expected squared norm of the difference between the model score and the true data score :

$$\mathbb{E}_{p(x)} [\|\nabla_x \log p_\theta(x) - \nabla_x \log p(x)\|^2]$$

However we often don't know the true data score, this is why Hyvärinen introduced an equivalent objective *implicit score matching* (ISM), that under some regularity assumptions, depends only on derivatives of the model's log-density :

$$\mathbb{E}_{p(x)} [\text{Tr}(\nabla_x^2 \log p_\theta(x)) + \frac{1}{2} \|\nabla_x \log p_\theta(x)\|^2]$$

With such a score estimator at hand, one could use it to sample from $p(x)$ via Langevin dynamics :

$$x_{t+1} = x_t + \tau \nabla_x \log p(x_t) + \sqrt{2\tau} z_t, \quad z_t \sim \mathcal{N}(0, I_d)$$

This yields asymptotically exact samples from $p(x)$, assuming perfect score estimation and small enough steps.

2.2 Limitations of Single-Scale Langevin Dynamics

In practice, two key issues arise when applying this strategy to high-dimensional data :

Manifold hypothesis. Real datasets often concentrate on a low-dimensional manifold embedded in $\mathbb{R}^d$. In such cases, the data distribution can be singular with respect to Lebesgue measure, so a density p on $\mathbb{R}^d$ may not exist and the score $\nabla_x \log p$ is not defined off the manifold in the usual sense. While score matching may work well on the data, Langevin chains initialized randomly in space receive uninformative or exploding gradients. Moreover, the ISM–ESM equivalence breaks when p is not strictly positive.

Multimodal distributions. Langevin dynamics is a local sampler : it follows the gradient of the log-density but cannot easily traverse low-density valleys between well-separated modes. To illustrate this, we consider a synthetic 2D Gaussian mixture with modes of varying weights and variances. In this specific setting, we have access to the exact score of the data distribution, which we use to sample via Langevin dynamics. Figure 1 illustrates that Langevin chains initialized from a uniform prior rapidly collapse to nearby modes, leading to poor exploration of the remaining ones (red crosses denote the true mode means). Quantitatively, the true mode weights are poorly recovered, as shown by the orange bars in Figure 4.

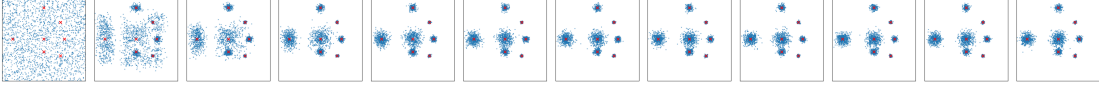


FIGURE 1 – Langevin dynamics from a uniform prior.

2.3 Noise-Conditioned Score Estimation and Annealed Sampling

To address both issues, Song and Ermon [4] proposed to estimate *noise-conditional scores* $\nabla_x \log p_\sigma(x)$ for a family of smoothed distributions p_σ , and sample progressively from high to low noise levels. Instead of targeting $p(x)$, the model learns a function $s_\theta(x, \sigma)$ trained via denoising score matching :

$$s_\theta(x, \sigma) \approx \nabla_x \log p_\sigma(x) \quad (1)$$

This resolves the manifold issue as p_σ has full support and enables more robust sampling using *Annealed Langevin Dynamics (ALD)*.

We discretize the range of noise scales into a decreasing sequence $\{\sigma_\ell\}_{\ell=1}^L$.

The index $\ell \in \{1, \dots, L\}$ denotes the current *noise level*, so that $x_t^{(\ell)}$ is the iterate at Langevin step t for noise scale σ_ℓ , and ϵ_ℓ is the step size used at that level :

$$x_{t+1}^{(\ell)} = x_t^{(\ell)} + \epsilon_\ell \nabla_x \log p_{\sigma_\ell}(x_t^{(\ell)}) + \sqrt{2\epsilon_\ell} z_t \quad (2)$$

At large σ , modes merge and particles can mix. As $\sigma \rightarrow 0$, the distribution sharpens and Langevin recovers high-fidelity details.

Moreover, In the Gaussian case, Vincent (2011) [6] shows that the score of the smoothed density can be learned by minimizing the conditional denoising score matching objective

$$\mathcal{L}_\sigma(\theta) = \mathbb{E}_{p(x)} \mathbb{E}_{\tilde{x} \sim \mathcal{N}(x, \sigma^2 I)} \left[\|s_\theta(\tilde{x}, \sigma) - \nabla_{\tilde{x}} \log p_\sigma(\tilde{x} | x)\|^2 \right]$$

For Gaussian corruption we have

$$p_\sigma(\tilde{x} | x) = \mathcal{N}(x, \sigma^2 I), \quad \nabla_{\tilde{x}} \log p_\sigma(\tilde{x} | x) = -\frac{\tilde{x} - x}{\sigma^2}$$

so that

$$\mathcal{L}_\sigma(\theta) = \mathbb{E}_{p(x)} \mathbb{E}_{\tilde{x} \sim \mathcal{N}(x, \sigma^2 I)} \left[\left\| s_\theta(\tilde{x}, \sigma) + \frac{\tilde{x} - x}{\sigma^2} \right\|^2 \right]$$

The minimizer of this loss satisfies

$$s_\theta(z) = \nabla_z \log p_\sigma(z)$$

This provides a practical way to estimate the score of p_σ without ever computing the true density or its second derivatives explicitly. It makes score estimation easier and well-defined off-manifold.

This is illustrated in Figures 2 and in the right panel of Figure 3, and by the green bars in Figure 4. Overall, ALD matches the true mixture proportions and avoids mode collapse.

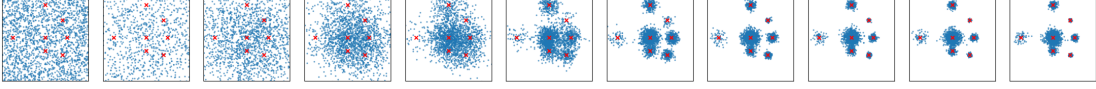


FIGURE 2 – Annealed Langevin dynamics from a uniform prior.

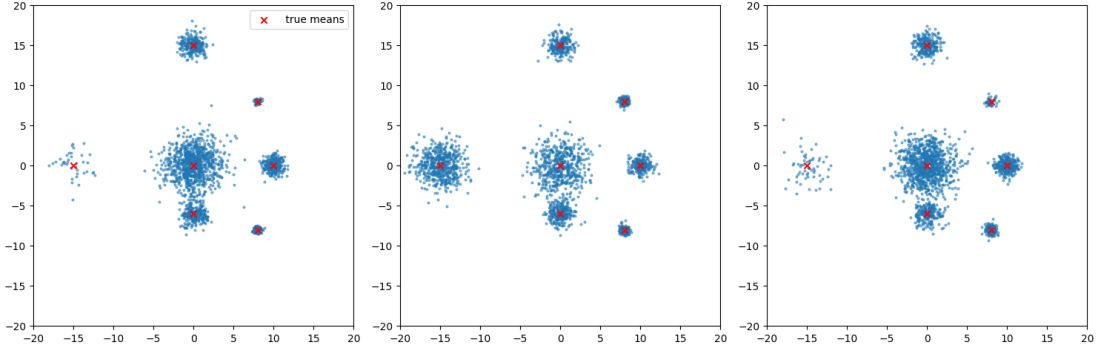


FIGURE 3 – Left : true samples. Center : Langevin samples. Right : ALD samples.

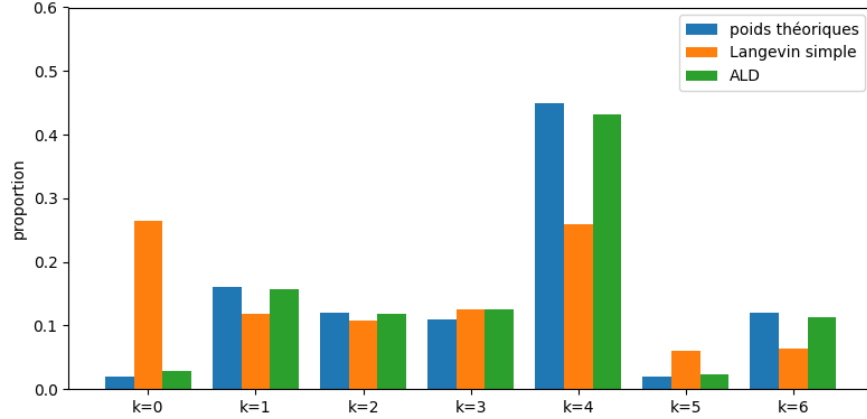


FIGURE 4 – Mode proportions : Langevin (orange) ; ALD (green) ; true weights (blue).

The combination of denoising score matching and annealed sampling culminated in the first *Noise Conditional Score Network* (NCSN), as introduced by Song and Ermon (2019) [4]. By estimating scores at multiple noise levels and using ALD, NCSNs overcame fundamental limitations of early score-based generative models.

In the next section, we describe how these ideas are in fact particular case of continuous time SDE forward and backward process.

3 Score-based generative modeling with SDEs

3.1 Forward and reverse SDEs

The continuous-time viewpoint of score-based models starts from a diffusion process

$$dx_t = f(x_t, t) dt + g(t) dw_t, \quad t \in [0, T]$$

where w_t is a standard Wiener process, $f : \mathbb{R}^d \times [0, T] \rightarrow \mathbb{R}^d$ is a prescribed drift, and $g : [0, T] \rightarrow \mathbb{R}$ is a scalar diffusion coefficient. We choose the SDE so that

$$x_0 \sim p_0 = p_{\text{data}}, \quad x_T \sim p_T$$

where p_T is a simple prior such as a centred Gaussian. The process induces a family of marginal densities $\{p_t\}_{t \in [0, T]}$, and for $0 \leq s < t \leq T$ a transition kernel

$$p_{st}(x_t | x_s)$$

that describes the distribution of x_t given x_s .

It has been shown [1] that this diffusion can be reversed in time. The reverse-time process $\{x_t\}_{t \in [0, T]}$ that maps p_T back to p_0 satisfies the SDE

$$dx_t = [f(x_t, t) - g(t)^2 \nabla_x \log p_t(x_t)] dt + g(t) d\bar{w}_t$$

where $\bar{w}_t$ is a standard Wiener process when time runs backwards. The only unknown term is the score function $\nabla_x \log p_t(x)$ for $t \in [0, T]$.

To approximate this score, we train a time-dependent neural network $s_\theta(x, t)$ with a continuous denoising score matching objective [5]. Let $x_0 \sim p_{\text{data}}$ and $x_t \sim p_{0t}(x_t | x_0)$ be obtained by simulating the forward SDE from time 0 to time t . The loss is

$$\mathcal{L}_{\text{SDE}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0, T]} \left[\lambda(t) \mathbb{E}_{x_0 \sim p_{\text{data}}} \mathbb{E}_{x_t \sim p_{0t}(\cdot | x_0)} \|s_\theta(x_t, t) - \nabla_{x_t} \log p_{0t}(x_t | x_0)\|_2^2 \right] \quad (3)$$

where $\lambda(t) > 0$ is a weighting function. For the VE SDE considered below, the forward transition kernel $p_{0t}(x_t | x_0)$ is Gaussian with closed-form mean and variance, which gives an explicit expression for $\nabla_{x_t} \log p_{0t}(x_t | x_0)$ and makes this objective practical to compute.

With enough capacity and data, the minimizer s_{θ^*} satisfies

$$s_{\theta^*}(x, t) = \nabla_x \log p_t(x) \quad \text{for almost all } (x, t)$$

so that we can plug s_{θ^*} into the reverse SDE and obtain a generative model that transforms Gaussian noise at time T into data-like samples at time 0.

3.2 Link between SMLD and the VE SDE

Introduce a continuous time variable $t \in [0, T]$ and a smooth variance schedule $\sigma(t)$ such that $\sigma(t_i) = \sigma_i$ for a grid $0 = t_0 < t_1 < \dots < t_N = T$. Writing $\Delta t_i = t_i - t_{i-1}$, the update above can be rewritten as

$$x_{t_i} = x_{t_{i-1}} + \sqrt{\sigma^2(t_i) - \sigma^2(t_{i-1})} z_{t_{i-1}} \approx x_{t_{i-1}} + \sqrt{\left. \frac{d}{dt} \sigma^2(t) \right|_{t=t_{i-1}}} \Delta t_i z_{t_{i-1}}$$

When the time step Δt_i goes to zero, the interpolated process converges to the Itô SDE

$$dx_t = g(t) dw_t, \quad g(t) = \sqrt{\frac{d}{dt} \sigma^2(t)}$$

which is referred to as the *Variance Exploding SDE (VE SDE)* because the marginal variance of x_t increases with t .

Conditionally on the starting point x_0 , the forward VE SDE has Gaussian transitions

$$x_t | x_0 \sim \mathcal{N}(x_0, \sigma^2(t) I_d), \quad \sigma(0) = 0$$

Hence its forward perturbation kernels coincide exactly with those of SMLD. Choosing a geometric schedule for $\{\sigma_i\}$ corresponds to a smooth function $\sigma(t)$ with exponentially increasing variance. In our implementation we use

$$\sigma(t) = \sigma^t, \quad t \in [0, 1]$$

for some $\sigma > 1$. In this case

$$\sigma^2(t) = \sigma^{2t} \quad \Rightarrow \quad \frac{d}{dt} \sigma^2(t) = 2 \log(\sigma) \sigma^{2t}$$

so that the diffusion coefficient becomes

$$g(t) = \sqrt{\frac{d}{dt} \sigma^2(t)} = \sqrt{2 \log(\sigma)} \sigma^t$$

Training an SDE-based model with the continuous-time objective $\mathcal{L}_{\text{SDE}}$ therefore extends SMLD from a finite ladder of noise levels to a continuum of times.

3.3 Solving the reverse SDE

Once the score network $s_{\theta^*}(x, t)$ has been trained, we build an approximation of the reverse-time SDE by replacing the true score with the neural network :

$$dx_t = [f(x_t, t) - g(t)^2 s_{\theta^*}(x_t, t)] dt + g(t) d\bar{w}_t$$

For the VE SDE associated with SMLD we have $f(x, t) = 0$, so the reverse dynamics simplify to

$$dx_t = -g(t)^2 s_{\theta^*}(x_t, t) dt + g(t) d\bar{w}_t$$

In practice we simulate this SDE on a discrete time grid

$$T = t_N > t_{N-1} > \dots > t_0 = 0$$

using a numerical solver. A simple choice is the Euler-Maruyama scheme, which gives the predictor update

$$x_{k-1} = x_k + \left[f(x_k, t_k) - g(t_k)^2 s_{\theta^*}(x_k, t_k) \right] \Delta t_k + g(t_k) \sqrt{|\Delta t_k|} z_k$$

with $\Delta t_k = t_{k-1} - t_k < 0$ and $z_k \sim \mathcal{N}(0, I_d)$. Initialization is done by sampling x_T from the prior p_T , for example $x_T \sim \mathcal{N}(0, \sigma^2(T) I_d)$ in the VE case.

To reduce discretization error and improve sample quality, [5] propose *predictor-corrector (PC)* samplers. At each time t_k , a predictor step is followed by a few Langevin corrector updates :

$$x^{(m+1)} = x^{(m)} + \varepsilon_k s_{\theta^*}(x^{(m)}, t_k) + \sqrt{2\varepsilon_k} z^{(m)}, \quad z^{(m)} \sim \mathcal{N}(0, I_d)$$

If the predictor is omitted, the resulting corrector-only (C-only) scheme reduces to annealed Langevin dynamics over a discrete noise schedule (SMLD/NCSN). In our project, VE-SDE therefore provides a direct comparison : C-only recovers SMLD, while PC augments it with reverse-SDE integration between noise levels.

4 Implementation and Results

We summarize our implementation choices and evaluation protocol. Qualitative results are deferred to the Appendix due to space constraints.

4.1 Scope and shared architecture

We first implemented the original NCSN [4] on a Gaussian mixture model, then on MNIST and CelebA, mainly to validate the pipeline and annealed Langevin dynamics in the discrete setting. In this report, we focus on VE-SDE.

All experiments share the same backbone : a U-Net conditioned on a single scalar (noise level or time), reused across datasets and formulations (discrete NCSN, VE-SDE). We train VE-SDE models on MNIST, CelebA 64×64 , and CIFAR-10 64×64 . (Some results Figures 7, B.3 and 6d.)

4.2 VE-SDE samplers : PC, C-only and P-only

A convenient aspect of training with the VE-SDE loss (3) is that it enables several samplers at inference time :

- **P-only** : predictor only, i.e. pure reverse-SDE integration.
- **C-only** : corrector only, which recovers an NCSN-style sampler at fixed noise levels.
- **PC** : predictor-corrector, applying both steps at each noise level.

As also reported in [5], we obtain the best results with PC. In Appendices B.1 and B.2, we compare these variants qualitatively on MNIST and CelebA 64×64 using the same trained VE-SDE model.

4.3 EMA on CIFAR-10

For CIFAR-10, we also evaluate the effect of an exponential moving average (EMA) of the network parameters at sampling time. EMA maintains a smoothed copy of the weights along training,

$$\theta_{\text{EMA}} \leftarrow \beta \theta_{\text{EMA}} + (1 - \beta) \theta,$$

which typically reduces variance in the learned score and improves generation quality. With the same trained model and the same PC sampler, we obtain $\text{FID}_{\text{no-EMA}} = 58.33$ versus $\text{FID}_{\text{EMA}} = 35.33$, showing improved sample quality. Qualitative samples and FID values are reported in Appendix B.3 (Figure 8).

4.4 SNR-based corrector

A key implementation choice is the corrector step size. At a given noise level σ , a Langevin corrector step is

$$x^{(k+1)} = x^{(k)} + \tau s_{\theta}(x^{(k)}, \sigma) + \sqrt{2\tau} z^{(k)}, \quad z^{(k)} \sim \mathcal{N}(0, I_d).$$

Rather than tuning τ by hand, we fix a target signal-to-noise ratio ρ and choose τ accordingly. This approach provides a more principled and intuitive control of the sampling dynamics by directly tuning the signal-to-noise ratio. Sample trajectories for different SNR values, starting from the same initial noise realization are shown in Figure 5 and additional details are given in Appendix A.

5 Discussion, perspectives, and conclusion

Our results validate the VE-SDE formulation as a robust framework for generative modeling, and we observed that scaling to higher resolutions benefits from stabilization via EMA. Future work could investigate semantic properties through image interpolation, or extend these diffusion processes to other modalities such as raw audio generation.

Bibliographie

- [1] Brian D. O. ANDERSON. “Reverse-time diffusion equation models”. In : *Stochastic Processes and their Applications*. T. 12. 1982.
- [2] Jonathan HO, Ajay JAIN et Pieter ABBEEL. “Denoising Diffusion Probabilistic Models”. In : *Advances in Neural Information Processing Systems*. T. 33. 2020, p. 6840-6851.
- [3] Aapo HYVÄRINEN. “Estimation of Non-Normalized Statistical Models by Score Matching”. In : *Journal of Machine Learning Research* 6 (2005), p. 695-709.
- [4] Yang SONG et Stefano ERMON. “Generative Modeling by Estimating Gradients of the Data Distribution”. In : *Advances in Neural Information Processing Systems*. T. 32. 2019.
- [5] Yang SONG et al. “Score-Based Generative Modeling Through Stochastic Differential Equations”. In : *International Conference on Learning Representations*. 2021.
- [6] Pascal VINCENT. “A Connection Between Score Matching and Denoising Autoencoders”. In : *Neural Computation* 23.7 (2011), p. 1661-1674.

Annexe

A SNR-based corrector

At each noise scale σ , our goal is to run a Langevin-type corrector step with a *fixed signal-to-noise ratio (SNR)* between the deterministic update and the injected Gaussian noise. This gives a direct handle on how “aggressive” the denoising step is at a given noise level, and it turned out to be much easier to tune in practice than manually specifying a schedule of step sizes for MCMC.

Definition of the SNR for the corrector. Given a current sample $x \in \mathbb{R}^d$ at noise level σ , a single corrector step is

$$x^{(k+1)} = x^{(k)} + \tau s_\theta(x^{(k)}, \sigma) + \sqrt{2\tau} z^{(k)}, \quad z^{(k)} \sim \mathcal{N}(0, I_d)$$

where $\tau > 0$ is the step size and s_θ is the learned score. We interpret

$$\Delta x_{\text{det}} = \tau s_\theta(x^{(k)}, \sigma) \quad \text{and} \quad \Delta x_{\text{noise}} = \sqrt{2\tau} z^{(k)}$$

as the “signal” and “noise” parts of the update.

We define the (squared) SNR of this update as

$$\text{SNR}^2(x^{(k)}, \sigma; \tau) = \frac{\|\Delta x_{\text{det}}\|_2^2}{\mathbb{E}\|\Delta x_{\text{noise}}\|_2^2} = \frac{\|\tau s_\theta(x^{(k)}, \sigma)\|_2^2}{2\tau \mathbb{E}\|z^{(k)}\|_2^2} = \frac{\tau \|s_\theta(x^{(k)}, \sigma)\|_2^2}{2d}$$

since $\mathbb{E}\|z^{(k)}\|_2^2 = d$ for $z^{(k)} \sim \mathcal{N}(0, I_d)$.

Fixing a target SNR $\rho > 0$ means imposing

$$\text{SNR}(x^{(k)}, \sigma; \tau) = \rho$$

which yields an adaptive step size

$$\tau(x^{(k)}, \sigma) = \frac{2d\rho^2}{\|s_\theta(x^{(k)}, \sigma)\|_2^2}$$

In practice, we use a batch-average version of this formula, but the principle is the same : the step size is chosen so that each corrector step has a prescribed SNR.

Gaussian case and scaling with σ . To build intuition, consider the ideal Gaussian case at a fixed noise scale σ , with

$$x = x_0 + \sigma z, \quad z \sim \mathcal{N}(0, I_d)$$

and the exact score

$$s^*(x, \sigma) = \nabla_x \log p_\sigma(x | x_0) = -\frac{x - x_0}{\sigma^2}$$

If x is highly noisy, the residual $x - x_0$ behaves like σz , so

$$s^*(x, \sigma) \approx -\frac{1}{\sigma} z, \quad \|s^*(x, \sigma)\|_2^2 \approx \frac{1}{\sigma^2} \|z\|_2^2 \approx \frac{d}{\sigma^2}$$

Plugging this into the SNR-based step size gives

$$\tau(x, \sigma) \approx \frac{2d\rho^2}{d/\sigma^2} = 2\rho^2\sigma^2$$

Thus, in the Gaussian regime, enforcing a fixed SNR *automatically* yields a step size proportional to σ^2 : large steps at high noise (where the target is heavily blurred) and small steps at low noise (where we only want to make fine corrections). This matches the usual heuristic schedules used in annealed Langevin dynamics, but here it follows directly from the SNR condition.

Effect of increasing or decreasing the target SNR. For images, the effect of the target SNR ρ is quite intuitive :

- If we *increase* ρ , the deterministic part Δx_{det} dominates the stochastic noise : each corrector step pushes the sample more strongly along the score direction, with relatively little added Gaussian noise. This can lead to faster denoising and sharper samples, but also a higher risk of getting stuck in poor modes or amplifying artefacts if the score estimate is imperfect.
- If we *decrease* ρ , the update becomes more noise-dominated : each step behaves more like a diffusive jitter around the current point. This tends to improve mixing and robustness (especially early in the chain), but denoising becomes slower and samples may remain overly smooth if ρ is too small.

In practice, we found that tuning a global SNR is much easier than hand-designing a step-size schedule, and it provides a more interpretable knob to balance exploration (noise) and denoising (signal) in the corrector.

To visualise this effect, it is most informative to look at the role of the SNR parameter in the C-only sampler. Figure 5 shows, for a fixed initial noise sample, the evolution of a single trajectory as we vary the target SNR from 0.05 (top row) to 1 (bottom row). This figure corresponds to the C-only sampler with $(N_{\text{steps}}, n_{\text{corr}}) = (2000, 1)$.

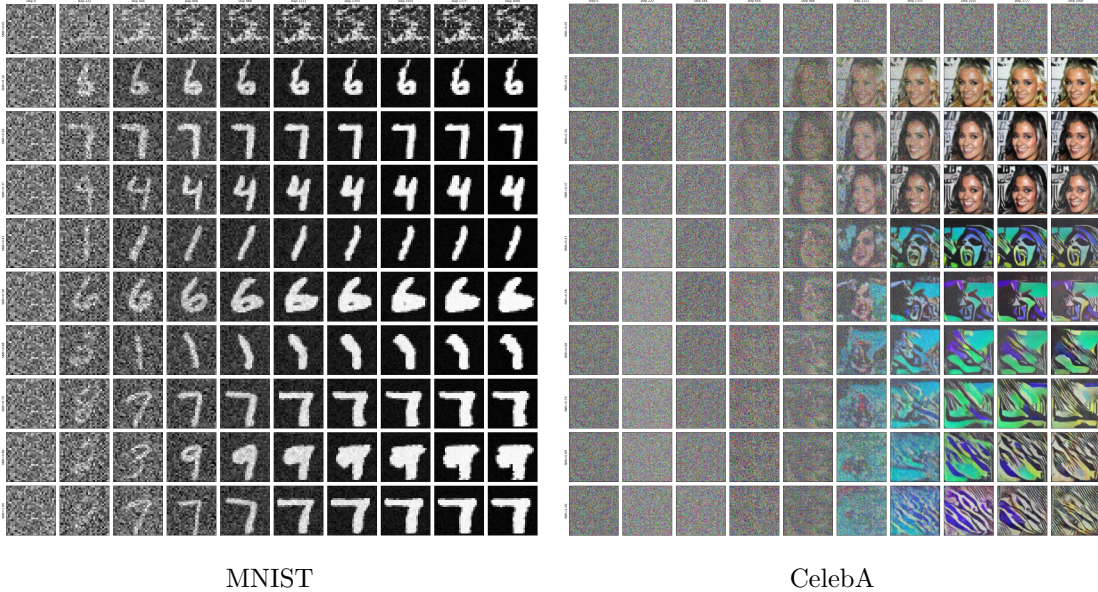


FIGURE 5 – C-only trajectories for different SNR values (MNIST and CelebA).

The effect is most visible on CelebA, where three regimes clearly emerge :

- **SNR too low** (~ 0.05). The trajectory remains almost pure noise until the end : the Langevin step is too small, and the “correction” is insufficient for a fixed step budget.
- **Intermediate SNR** (~ 0.16 – 0.37). A face gradually emerges and becomes sharper.
- **SNR too high** ($\geq \sim 0.47$). The dynamics drift toward high-frequency patterns/textures (striping, flat regions), and the face quality degrades.

B Visual result

B.1 PC, C-only and P-only samplers

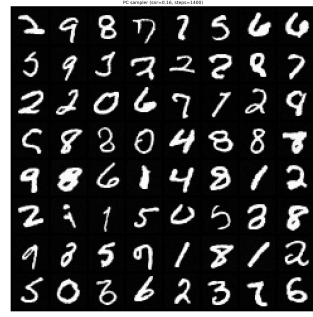
Figure 6 complements the discussion in Section 4.2 by comparing PC, C-only and P-only samplers on MNIST/CelebA.



(a) MNIST, PC sampler



(b) MNIST, C-only sampler



(c) MNIST, P-only sampler



(d) CelebA, PC sampler



(e) CelebA, C-only sampler



(f) CelebA, P-only sampler

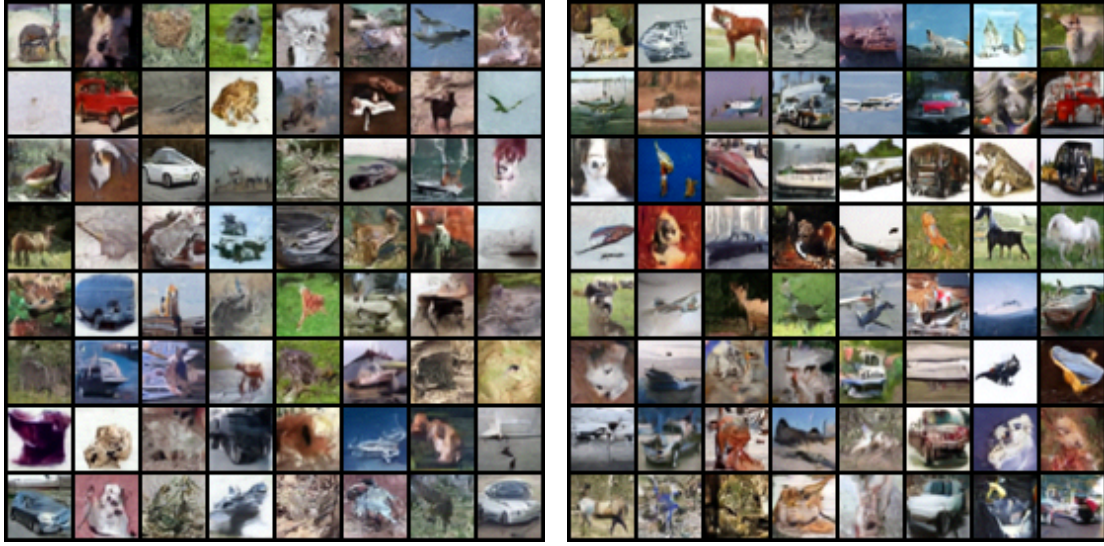
FIGURE 6 – Comparison of PC, C-only and P-only samplers on MNIST (top row), CelebA (bottom row).

B.2 PC samples on CelebA



FIGURE 7 – PC trajectories for CelebA 64x64

B.3 Impact of EMA on CIFAR-10 samples



No EMA (FID = 58.33)

EMA (FID = 35.33)

FIGURE 8 – CIFAR-10 samples generated with the same VE-SDE model, fixed SNR, and PC sampler, without EMA (left) and with EMA (right).